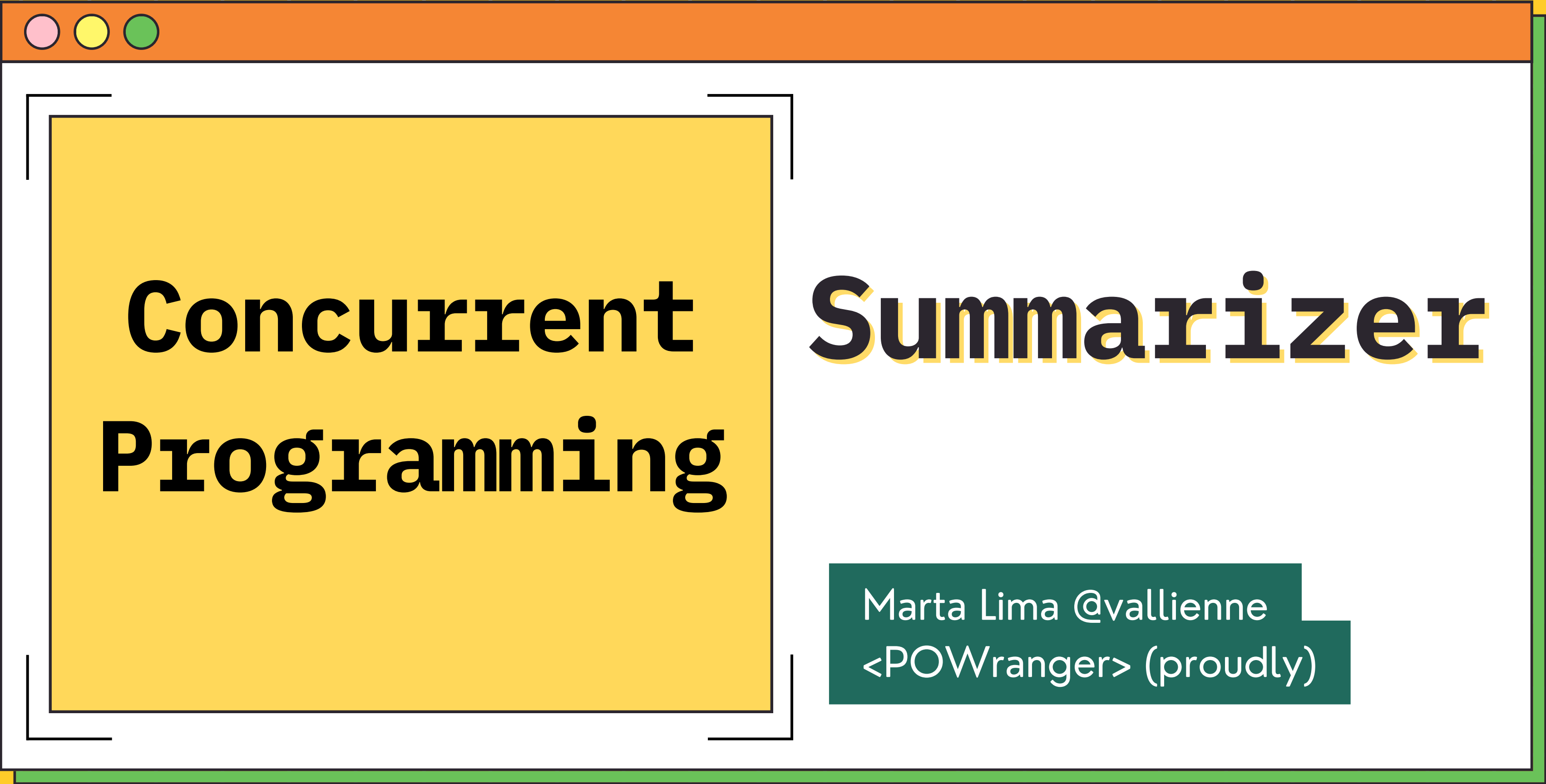




Concurrent Programming

Summarizer

Marta Lima @vallienne
<POWranger> (proudly)



How I wanted to be



How I am



Mission: Thread Synchronization

"some threads just don't like to
wait!"

Lock?



There is no Heaven



Who gets the Lock?

Every Thread wants it!!!!

Threads acquire a lock
when entering a
synchronized block

Threads release a lock when
exiting the synchronized block

Keyword Synchronized

Blocks the access
to the method of
a determined
Object

- Sometimes we don't want to synchronize the **entire method**, only some instructions within it.
- We can achieve this by applying synchronized to a **block**

THREAD SYNCH WITH GUARDED BLOCKS WHEN THERE IS A **CONDITION**

`wait()`

The method is put on hold
until the condition is verified!
(passes the Lock)

`notify()` `notifyAll()`

There has to be a way that
unblocks the method

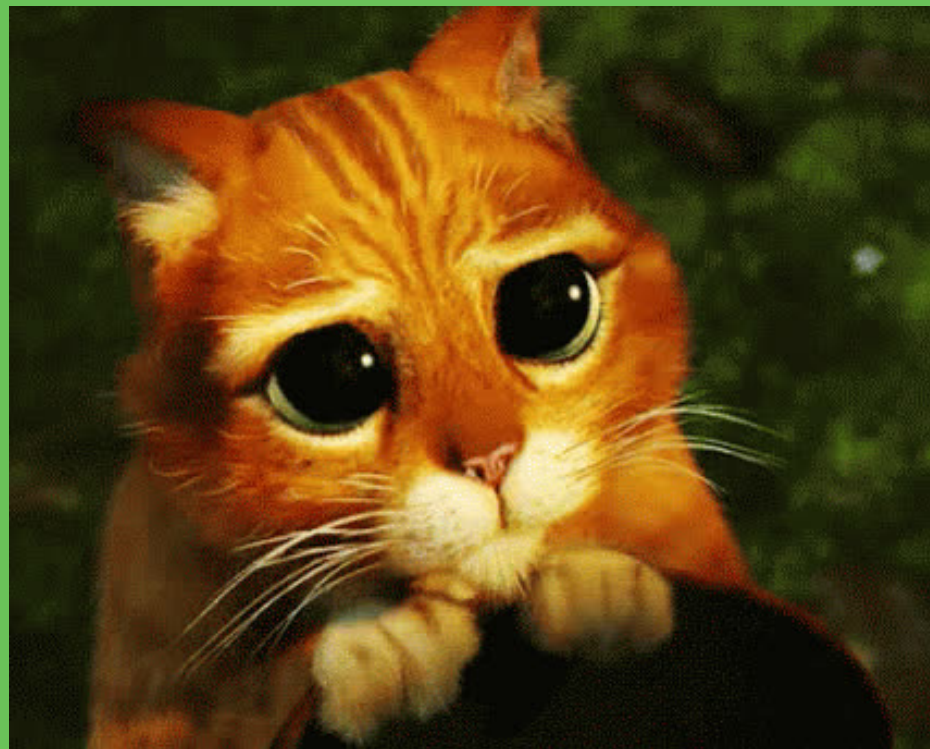
OR!!!!

There will be a...

DEADLOCK!!!!

My Beautiful Code

Please be nice!!!



Container with shared info that has to be protected

```
public class BQueue<T> {  
    private int limit;  
    Queue<T> bake;  
  
    public BQueue(int limit) {  
        this.limit = limit;  
        bake = new LinkedList<>();  
    }  
}
```

The outcome

```
A pizza has been added to the counter, there are 1 pizzas available  
Someone just took a pizza@ pizzas availableA PEPPERS pizza  
A pizza has been added to the counter, there are 1 pizzas available  
Joana baked a A PEPPERS pizza  
Someone just took a pizza@ pizzas availableA EXTRA_CHEESE pizza  
Tomás María ate a A PEPPERONI pizza  
Rodrigo José ate a A PEPPERONI pizza  
Silverio baked a A EXTRA_CHEESE pizza  
A pizza has been added to the counter, there are 1 pizzas available  
Joana baked a A PEPPERS pizza  
A pizza has been added to the counter, there are 2 pizzas available  
Silverio baked a A EXTRA_CHEESE pizza  
Someone just took a pizza@ pizzas availableA PEPPERS pizza  
Tomás María ate a A PEPPERONI pizza  
Someone just took a pizza@ pizzas availableA EXTRA_CHEESE pizza  
Rodrigo José ate a A PEPPERONI pizza  
A pizza has been added to the counter, there are 1 pizzas available  
Joana baked a A PEPPERS pizza  
Someone just took a pizza@ pizzas availableA PEPPERS pizza  
A pizza has been added to the counter, there are 1 pizzas available  
Tomás María ate a A PEPPERONI pizza  
Someone just took a pizza@ pizzas availableA EXTRA_CHEESE pizza  
Silverio baked a A EXTRA_CHEESE pizza  
Rodrigo José ate a A PEPPERONI pizza  
A pizza has been added to the counter, there are 1 pizzas available  
Silverio baked a A EXTRA_CHEESE pizza  
Tomás María ate a A PEPPERONI pizza  
Someone just took a pizza@ pizzas availableA EXTRA_CHEESE pizza  
A pizza has been added to the counter, there are 1 pizzas available  
Joana baked a A PEPPERS pizza  
Someone just took a pizza@ pizzas availableA PEPPERS pizza  
Rodrigo José ate a A PEPPERONI pizza  
A pizza has been added to the counter, there are 1 pizzas available  
Someone just took a pizza@ pizzas availableA EXTRA_CHEESE pizza  
Tomás María ate a A PEPPERONI pizza  
Silverio baked a A EXTRA_CHEESE pizza  
A pizza has been added to the counter, there are 1 pizzas available  
Joana baked a A PEPPERS pizza  
Someone just took a pizza@ pizzas availableA PEPPERS pizza  
Rodrigo José ate a A PEPPERONI pizza  
A pizza has been added to the counter, there are 1 pizzas available  
Silverio baked a A EXTRA_CHEESE pizza  
Someone just took a pizza@ pizzas availableA EXTRA_CHEESE pizza  
Tomás María ate a A PEPPERONI pizza  
A pizza has been added to the counter, there are 1 pizzas available  
Joana baked a A PEPPERS pizza  
Someone just took a pizza@ pizzas availableA PEPPERS pizza  
Rodrigo José ate a A PEPPERONI pizza  
A pizza has been added to the counter, there are 1 pizzas available  
Silverio baked a A EXTRA_CHEESE pizza  
Someone just took a pizza@ pizzas availableA EXTRA_CHEESE pizza  
Tomás María ate a A PEPPERONI pizza  
A pizza has been added to the counter, there are 1 pizzas available  
Someone just took a pizza@ pizzas availableA PEPPERS pizza  
Joana baked a A PEPPERS pizza  
Rodrigo José ate a A PEPPERONI pizza
```



Here you can see the Object and one of the created Threads (MultiThreading)



```
BQueue<Pizza> queue= new BQueue<>(5);  
  
    Producer p1 = new Producer(queue,10);  
    Thread t1 = new Thread(p1);  
    t1.setName("Silvério");
```



Offer was the act of making the pizza **BUT ONLY IF** there was space left in the counter(list limit)



```
public synchronized void offer(T data) {  
    while (bake.size() >= limit) {  
        try {  
            wait();  
        } catch (InterruptedException e) {  
            throw new RuntimeException(e);  
        }  
    }  
    bake.offer(data);  
    if (bake.size() < limit)  
        notifyAll();  
    System.out.println("A pizza has been added to the counter, there are " + bake.size() + " pizzas  
available");  
}
```



Poll was the act of taking the pizza out **BUT ONLY IF** there were any pizzas in the counter (populated list?)



```
public synchronized T poll() {  
    if (bake.size()==0) {  
        try {  
            wait();  
        } catch (InterruptedException e) {  
            throw new RuntimeException(e);  
        }  
    }  
    T baked = bake.poll();  
    //System.out.println("Someone just took a pizza" + bake.size() + " pizzas available");  
  
    if (!bake.isEmpty())  
        notifyAll();  
    System.out.println("Someone just took a pizza" + bake.size() + " pizzas available" + baked);  
    return baked;  
}
```



Consumer ate the Pizzas. He is the one who calls the
SYNCHRONIZED METHOD poll()



```
public class Consumer implements Runnable {  
    @Override  
    @Override  
    public void run() {  
        Pizza pizza = new Pizza();  
        while (pizzasEaten != elementNum){  
            try {  
                Thread.sleep(1000);  
            } catch (InterruptedException e) {}  
            }  
        queue.poll();  
        System.out.println(Thread.currentThread().getName() + " ate a " + pizza.toString());  
    }  
}
```



Producer makes the Pizzas. He is the one who calls the
SYNCHRONIZED METHOD offer()



```
public class Producer implements Runnable {
    @Override
    public void run() {
        Pizza pizza = new Pizza();
        for (int i = 0; i < elementNum; i++) {
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
            queue.offer(pizza);
            bakedPizzas++;

            System.out.println(Thread.currentThread().getName() + " baked a " + pizza.toString());
        }
    }
}
```

THREAD POOLS AND EXECUTORS

Instead of
creating a new
Thread to every
new request...
(consume
problems... and
limits)

THE EXECUTOR FRAMEWORK

Share resources.
Reutilizes

COLLECTION OF THREADS

Perform Tasks
When "used" they don't
just "die".
They are Reutilized!

NOT SAFE AT ALL





POOL IMPLEMENTATION

Executor Interface

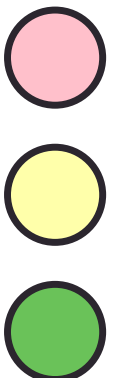
creates a Pool

Reuses existing threads
reduces the costs of creating
a new one for each request

EXECUTOR SERVICE

submits the several tasks of
the threads to execute

Has a shut down system tht
ejects new tasks





Creating a new thread pool

Fixed Thread Pool

fixed
max pool size

Cached Thread Pool

flexible
add on demand



THE END!

